Guía de arranque y pruebas

Prueba técnica · AI Engineer Challenge: Intelligent Notification Service (Python / FastAPI)

Esta guía explica, paso a paso y en Windows (PowerShell), cómo levantar el entorno del reto, arrancar el servicio, probarlo a mano, lanzar el test de carga con su scorecard en Grafana y ejecutar los tests automáticos.

Componente	Qué es	Puerto
app	Este servicio (FastAPI): recibe peticiones en lenguaje natural, extrae la notificación con IA y la envía al provider.	5000
provider	Mock del reto: motor de IA (`/v1/ai/extract`) y notificaciones (`/v1/notify`).	3001
grafana	Scorecard del test de carga.	3000
influxdb	Métricas de k6 y del provider.	8086

Contenido

1. Requisitos
2. Obtener el código
3. Paso 1 · Arrancar con Docker
4. Paso 2 · Probar el servicio a mano
5. Paso 3 · Test de carga y scorecard
6. Tests automáticos
7. Ejecución en local sin Docker (opcional)
8. Configuración
9. Problemas frecuentes
10. Parar todo

1. Requisitos

Herramienta	Para qué	Comprobar con
Git	Clonar el repositorio	`git --version`
Docker Desktop	Arrancar el servicio, el provider, InfluxDB, Grafana y k6. Debe estar abierto y con el motor arrancado (*Engine running*)	`docker info`
Python **3.12**	Sólo para los tests automáticos o para ejecutar sin Docker. Usa la 3.12: con versiones más nuevas algunas dependencias fijadas no tienen paquetes precompilados	`py -3.12 --version`

> **Nota.** No hace falta instalar k6: el test de carga se ejecuta en un contenedor de Docker.

2. Obtener el código

```
cd C:\Users\<usuario>\Documents
git clone https://github.com/ridaTDS/prueba-itx-ai-python.git
cd prueba-itx-ai-python
```

3. Paso 1 · Arrancar con Docker

1. Levanta la infraestructura del reto (provider, InfluxDB y Grafana):

```
docker-compose up -d provider influxdb grafana
```

2. Construye y arranca el servicio:

```
docker-compose up -d --build app
```

3. Comprueba que responde:

```
Invoke-RestMethod http://localhost:5000/health
```

Debe devolver `status: ok` con el número de peticiones por estado. La documentación interactiva de la API está en `http://localhost:5000/docs` y la del provider en `http://localhost:3001/docs`.

> **Nota.** Si usas una versión reciente de Docker, `docker compose` (sin guion) funciona igual.

4. Paso 2 · Probar el servicio a mano

El flujo tiene tres pasos: crear la petición, pedir que se procese y consultar su estado. En PowerShell:

```
# 1. Crear la petición -> 201 {"id": "..."}
$r = Invoke-RestMethod -Method Post -Uri http://localhost:5000/v1/requests `
       -ContentType 'application/json' `
       -Body '{"user_input": "Manda un correo a marta@app.com: Tienes un nuevo mensaje"}'

# 2. Procesarla -> 202 {"id": "...", "status": "queued"} (responde al momento)
Invoke-RestMethod -Method Post -Uri "http://localhost:5000/v1/requests/$($r.id)/process"

# 3. Consultar el estado (la IA tarda 1,5-3 s)
Start-Sleep -Seconds 5
Invoke-RestMethod "http://localhost:5000/v1/requests/$($r.id)"
```

Petición (`user_input`)	Estado final esperado
Manda un correo a marta@app.com: Tienes un nuevo mensaje	`sent` (email)
SMS al 600111222: reserva confirmada	`sent` (sms)
Avisa a Pedro de que llego tarde	`failed`: no hay email ni teléfono, no se reintenta

Estados posibles: `queued` (recibida) → `processing` (extrayendo con la IA o esperando turno en el provider) → `sent` o `failed`.

Otras comprobaciones

Prueba	Resultado esperado
Repetir `POST /process` sobre la misma petición	202 y un único envío (idempotente)
`GET /v1/requests/no-existe`	404
`POST /v1/requests` con `{"user_input": ""}`	422 (entrada vacía)
`GET /health`	Peticiones por estado, llamadas a la IA por petición y envíos pendientes

5. Paso 3 · Test de carga y scorecard

Con todo arrancado (paso 1), lanza el test oficial del reto. Dura unos 40 segundos y llega a 200 usuarios virtuales:

```
docker-compose run --rm load-test
```

Resultados en Grafana: `http://localhost:3000/d/ia-performance-scorecard/`.

> **Nota.** Elige un rango de tiempo que cubra todo el test (por ejemplo *Last 5 minutes*, arriba a la derecha) y pulsa *Refresh*. Con un rango de pocos segundos los paneles muestran datos parciales o «No data».

Panel	Qué mide	Valor esperado
Disponibilidad global / Estado resiliencia	% de peticiones sin error	100 % · ALTA
1. Respuesta de interfaz	Latencia media de `/process`	RÁPIDA (≈1-2 ms)
2. Rendimiento	Peticiones por segundo y latencia	≈200 req/s · latencia estable
4. Estabilidad p95/p99	Percentiles de `/process`	Pocos milisegundos
Concurrencia máxima	Usuarios virtuales de k6	≈200
Ratio intentos/req	Llamadas a la IA por `/process`	≈1,1 (sólo se reintentan los rechazos)

El provider sólo acepta 50 notificaciones cada 10 s. Durante el test la mayoría de peticiones quedan en `processing` esperando turno y se envían después a ~4 por segundo: es el comportamiento previsto. La evolución se ve en `GET /health` (`pending_notifications`).

6. Tests automáticos

Se ejecutan con Python 3.12, sin Docker. Desde la raíz del repositorio:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r app/requirements-dev.txt

pytest                          # todos (77 tests, ~10 s)
pytest -m "not integration"     # sólo unitarios y de API
pytest tests/test_guardrails.py -v   # un fichero concreto, con detalle
```

Fichero	Qué cubre
`test_guardrails.py`	Cada variante de respuesta del mock (Markdown, JSON truncado, comillas simples, claves alternativas, campos que faltan, rechazos), anclaje del destinatario y una prueba con 5.000 respuestas generadas por el propio mock.
`test_prompt.py`	Estructura de los mensajes, ejemplos few-shot, delimitadores y corrección guiada.
`test_extractor.py`	Presupuesto de llamadas a la IA, reintento guiado, fallo rápido y errores técnicos.
`test_ai_client.py` `test_sender.py`	Clientes HTTP: cabeceras, `trace_id`, 429, 5xx, 4xx y errores de red.
`test_rate_limiter.py`	Nunca más de N envíos por ventana de tiempo (con reloj simulado).
`test_pipeline_and_api.py`	Contrato de la API, idempotencia, estados finales, validación y `/health`.
`test_integration_provider.py`	Flujo completo contra el **provider real del reto**, servido en memoria (marcado como `integration`).

> **Nota.** Si PowerShell no deja ejecutar `Activate.ps1`, ejecuta antes `Set-ExecutionPolicy -Scope Process Bypass` (sólo afecta a esa ventana).

7. Ejecución en local sin Docker (opcional)

Con el entorno virtual del apartado 6 activado, en dos terminales:

```
# Terminal 1: provider del reto
cd provider
uvicorn app:app --port 3001

# Terminal 2: este servicio
cd app
uvicorn main:app --port 5000
```

> **Importante.** Sin Docker, el provider intenta escribir sus métricas en un InfluxDB que no existe y cada llamada a la IA se bloquea unos cientos de milisegundos. Sirve para probar el flujo a mano, pero para medir rendimiento usa Docker (apartados 3 y 5).

8. Configuración

Variables de entorno opcionales (valores por defecto en `app/notification_service/config.py`):

Variable	Por defecto	Descripción
`PROVIDER_BASE_URL`	`http://localhost:3001`	URL del provider (IA y notificaciones)
`PROVIDER_API_KEY`	`test-dev-2026`	Cabecera X-API-Key del provider

Variable	Por defecto	Descripción
`AI_MAX_ATTEMPTS`	3	Llamadas máximas a la IA por petición
`AI_MAX_CONCURRENCY`	100	Extracciones simultáneas
`NOTIFY_RATE_LIMIT`	40	Envíos por periodo (el provider admite 50)
`NOTIFY_RATE_PERIOD_SECONDS`	10	Periodo del límite de envíos
`NOTIFY_WORKERS`	20	Workers de envío
`NOTIFY_MAX_ATTEMPTS`	6	Intentos máximos por notificación

9. Problemas frecuentes

Síntoma	Causa	Solución
`open //./pipe/dockerDesktopLinuxEngine` al usar docker	Docker Desktop no está arrancado	Ábrelo y espera a «Engine running»; comprueba con `docker info`.
El puerto 5000 o 3000 ya está en uso	Otra aplicación (o la prueba de back, que también usa el 5000 y el 3000)	Para la otra aplicación o sus contenedores (`docker compose down` en su carpeta).
`pip install` falla compilando `pydantic-core`	Versión de Python demasiado nueva	Crea el entorno con Python 3.12: `py -3.12 -m venv .venv`.
`pytest` no encuentra `notification_service`	No se ejecuta desde la raíz del repositorio	Ejecuta `pytest` en la carpeta que contiene `pytest.ini`.
Grafana muestra «No data» o valores raros	Rango de tiempo demasiado corto o fuera del test	Elige *Last 5 minutes* y pulsa *Refresh* (apartado 5).
Tras el test hay muchas peticiones en `processing`	Esperan turno por el límite del provider	Es lo previsto: se envían a ~4/s; mira `/health`.

10. Parar todo

```
docker-compose down
```

Si ejecutaste en local sin Docker, para cada servidor con `Ctrl + C` en su terminal.